



Audio, UI, Navigazione e Animazioni

Massimiliano Pascoli, PhD Student

Human-Computer Interaction Laboratory

Department of Mathematics, Computer Science, and Physics

University of Udine - Italy

Programma (o almeno ci proviamo...)

- UNIUD Game Jam Crash Course Rewind
 - Recap generale argomenti + domande
 - Cosa avevamo (lo showcase)
 - Cosa avevamo (il dietro le quinte)
- Risultato Finale (lo showcase)
- Audio
 - Motivazioni x2
 - Componenti principali x4
 - Editor
- Interfaccia Utente
 - Motivazioni
 - Componenti principali x5
 - Editor
 - Proviamo insieme (Esercizio 1)
 - *Menù*
 - *Cambio scena*
- *Passaggio dati*
- Navigazione
 - Motivazioni
 - Componenti principali x5
 - Editor
 - Proviamo insieme (Esercizio 2)
 - *Macchina a stati finiti*
 - *Movimento di un NPC*
- Animazioni
 - Motivazioni
 - Possibili alternative
 - Componenti principali x5
 - Editor
 - Proviamo insieme (Esercizio 3)
 - *NPC animato*
 - *Rumore di camminata*
- Siti utili x2

Programma (o almeno ci proviamo...)

- Esercizio + Recap Finale (se c'è tempo)
 - Risultato finale (il dietro le quinte)
 - Proviamo insieme
 - Damage Indicator
 - Vari ed eventuali...

MA PRIMA....

- Create un progetto vuoto (3D-URP)

- Installate TextMeshPro

`Window > TextMeshPro > Import TMP Essential Resources`

- Caricate il .unitypackage (Lezione 3)

>> Potrebbe essere necessario un riavvio del progetto Unity <<

- Includete tutte le scene in `Scenes > Lezione 3` nella lista di scene incluse nella build

`File > Build Profiles > Open Scene List`

Audio

A vintage-style microphone on a stand is positioned in the foreground, slightly to the right. Behind it, a large mixing console with numerous sliders and knobs is visible, though slightly out of focus. The scene is dimly lit with warm, golden-brown tones, creating a professional and artistic atmosphere.

Audio: motivazioni (1)

- I videogiochi non sono solo grafica
- L'audio può
 - essere parte delle meccaniche di gioco
 - fornire feedback al giocatore
 - influenzare l'atmosfera
 - suscitare emozioni
 - distrarre dall'obiettivo
- Per utilizzarlo, bisogna considerare:
 - il tipo di audio (TBE)
 - formato e proprietà
 - ambiente in cui viene riprodotto



Audio: motivazione (2)

- Tipi di audio

- Vocalizzazione

- Dialoghi
 - NPC che parlano fra loro o con il giocatore, voce del giocatore in prima persona



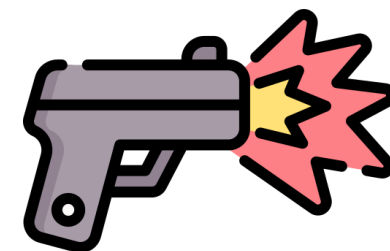
- Musica

- Suscitano emozioni o le enfatizzano
 - Colonne sonore



- SFX

- Solitamente danno un feedback
 - UI click, spari, ...



- Ambientale

- Creano atmosfera, suoni distanti
 - Pioggia, uccelli, vento, vociare di folle, ...



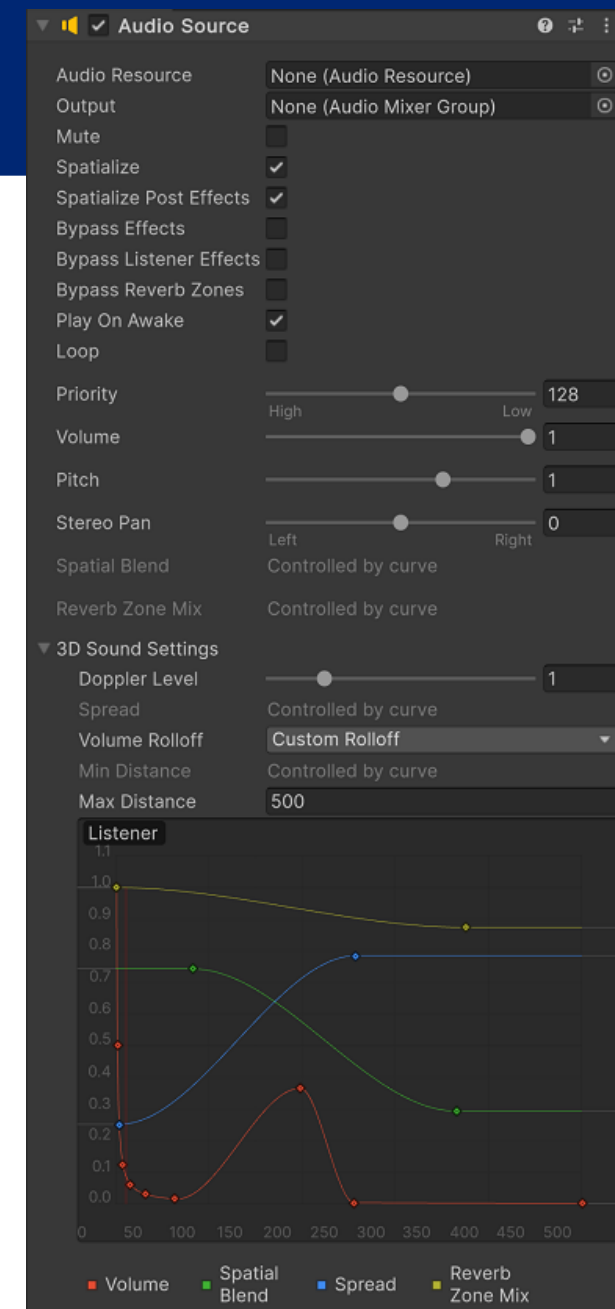
Audio: componenti principali (1)

- Audio source

- Qualsiasi cosa emetta un suono di qualsiasi tipo da una posizione (attaccato a un Game Object)
- L'output viene passato ad un mixer o direttamente a un Audio Listener

- Parametri importanti:

- Audio resource
- Output
- Play on Awake ([Info plz](#))
- Loop
- Volume
- Pitch
- Spatial blend
- Rolloff



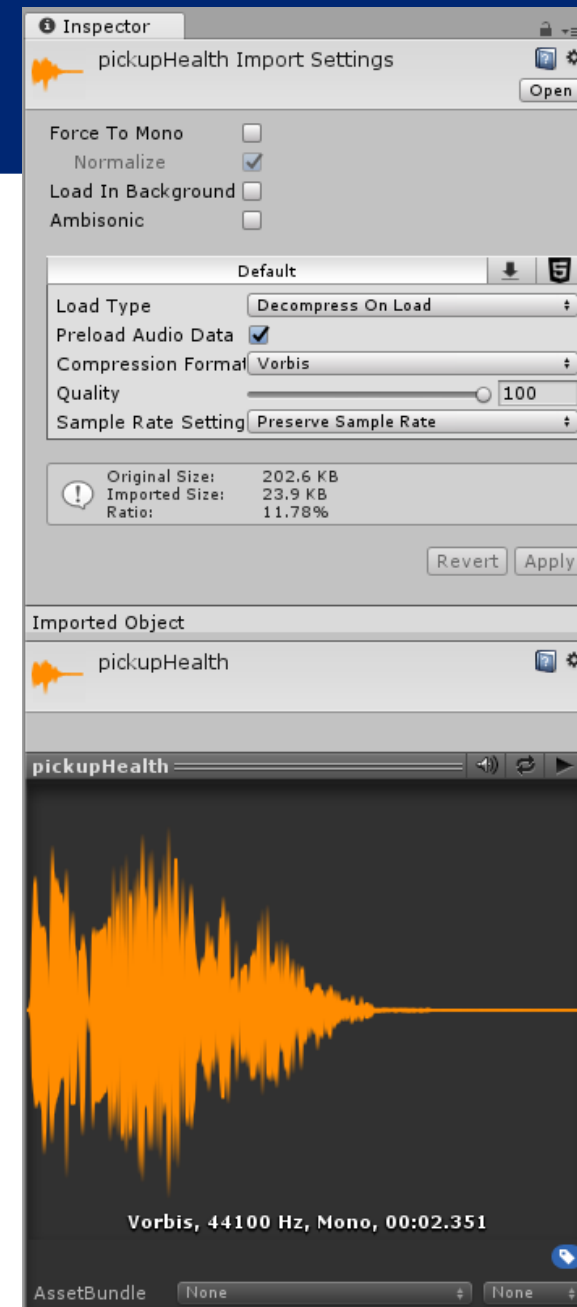
Audio: componenti principali (2)

- Audio Clip

- Se l'Audio Source sono degli altoparlanti attaccati ad un giradischi, la Audio Clip è il CD che volete ascoltare
- Bilanciare compressione, qualità, performance di caricamento (gestibile da parametri e da script)

- Audio listener

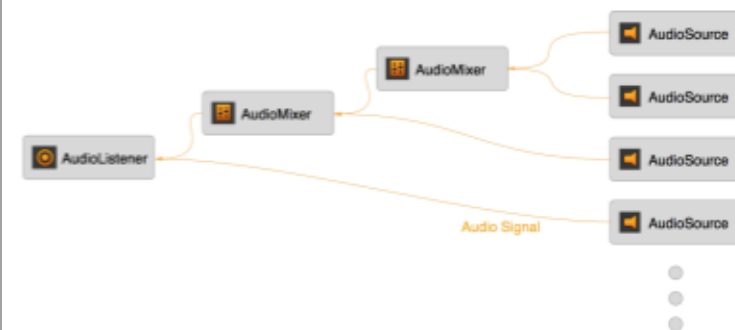
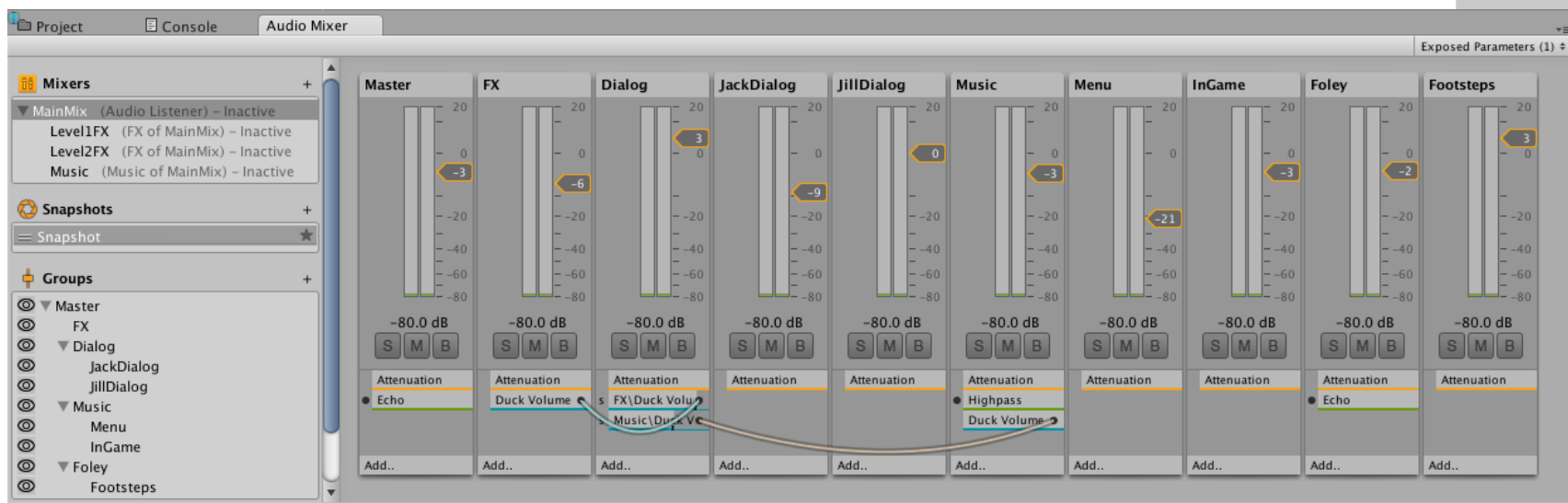
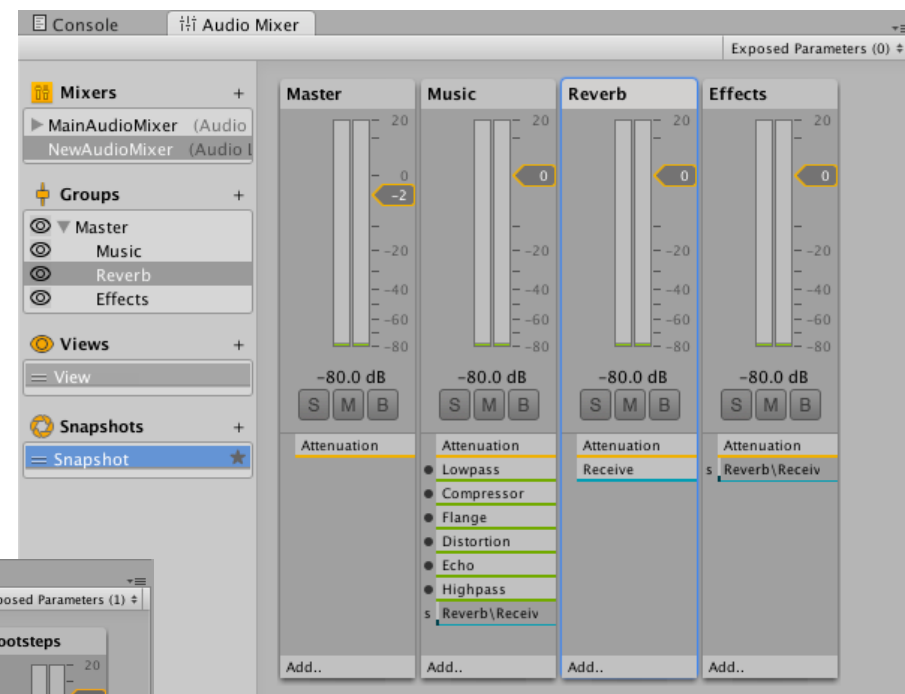
- Qualcuno deve pur ascoltare le Audio Source o gli Audio Mixer... :/
- Sono l'equivalente delle orecchie di un ascoltatore o di un microfono che ascolta tutto l'ambiente
- Nessun parametro visibile in editor (da script qualcosa si vede)



Audio: componenti principali (3)

• Audio mixer

- Per bilanciare le varie Audio Source e non solo (Ducking)
- Organizzano le Audio Source in alberi
- Pro Tip: un gruppo/mixer per tipo d'audio
 - Tipicamente almeno: Music Group e SFX Group + Master Group (volume generale)
- Per le modifiche da script ai parametri di mixing, bisogna esporre i parametri



Audio: componenti principali (4)

- Audio Filters

- Applicano effetti a Audio Source e audio in arrivo all'Audio Listener

Audio Filters

- Audio Low Pass Filter
- Audio High Pass Filter
- Audio Echo Filter
- Audio Distortion Filter
- Audio Reverb Filter
- Audio Chorus Filter

Audio Effects

- Audio Low Pass Effect
- Audio High Pass Effect
- Audio Echo Effect
- Audio Flange Effect
- Audio Distortion Effect
- Audio Normalize Effect
- Audio Parametric Equalizer Effect
- Audio Pitch Shifter Effect
- Audio Chorus Effect
- Audio Compressor Effect
- Audio SFX Reverb Effect
- Audio Low Pass Simple Effect
- Audio High Pass Simple Effect

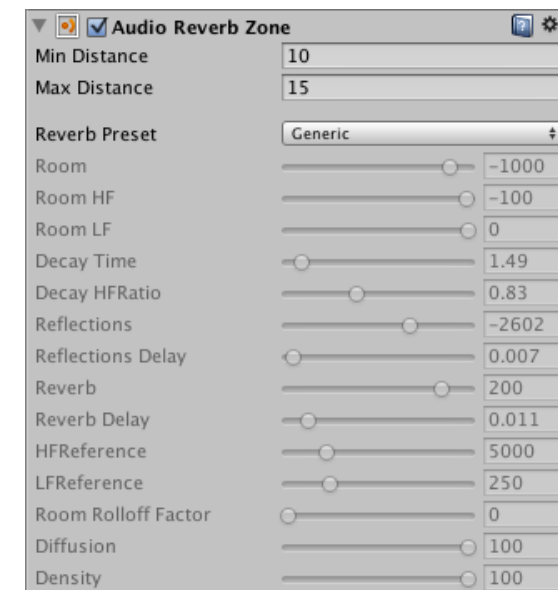
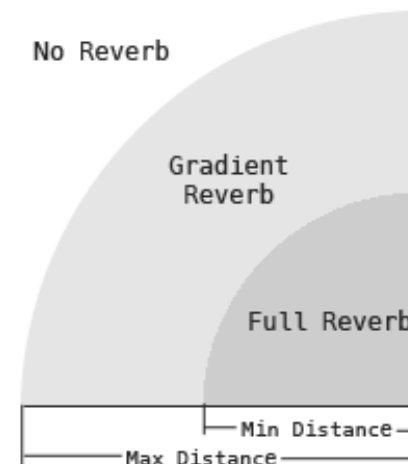
- Audio Effects

- Applicano effetti agli Audio Mixer Group

- Audio Reverb Zones

- Unity non calcola riverbero audio in base alla geometria, ma possiamo simularlo (ci sono pacchetti sperimentali per il raytracing audio)

Sappiate che esistono, ma per la Game Jam sono importanti
Clip, Listener e Source
Tempo ne avrete poco!!!





Interfaccia Utente

(G)UI: motivazioni

- L'interfaccia grafica per l'utente è ciò che distingue un film da un videogioco
- Consentono l'interazione del giocatore con il mondo di gioco

ATTENZIONE: La UI non è solo GUI, con interfaccia intendiamo anche altri tipi di interfaccia oltre a bottoni, finestre, ... (es. movimenti fisici)



UI : componenti principali (1)

- 2 (+1) modi per costruire GUI in Unity:
 - UI toolkit
 - Per la GUI di gioco
 - Unity User Interface (uGUI)
 - Per la GUI di gioco
 - Immediate Mode Graphical User Interface (IMGUI)
 - Solitamente per la GUI dell'editor

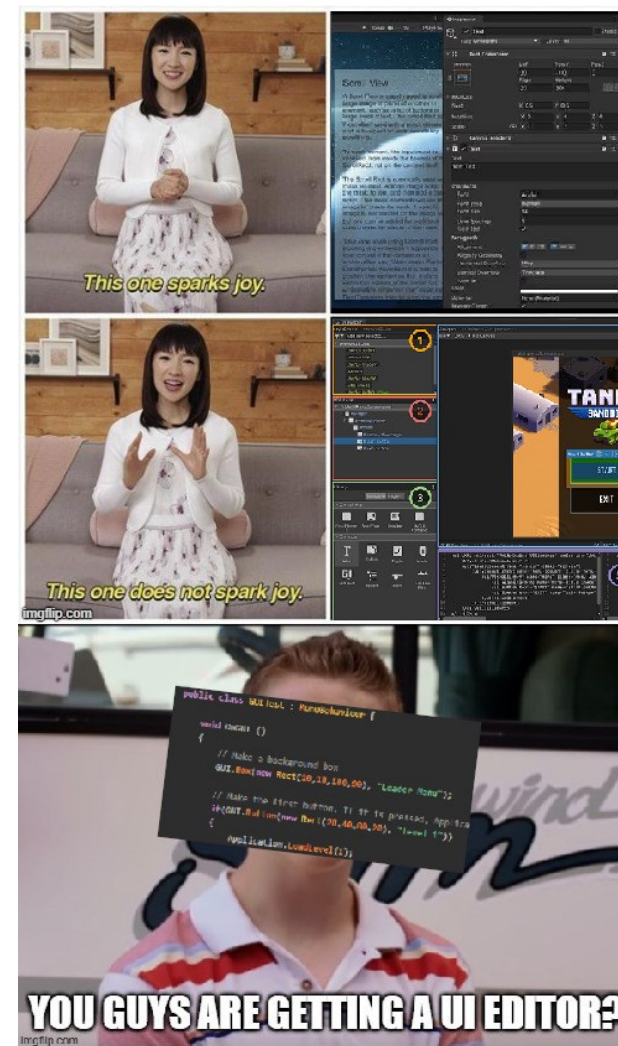
ATTENZIONE: Alla Game Jam potete usare il sistema che preferite, MA non riceverete supporto per UI toolkit e non lo tratterò nel corso

Usate
TextMeshPro

Game Object -> transform

vs

UI Object -> rect transform
(?differenze?)



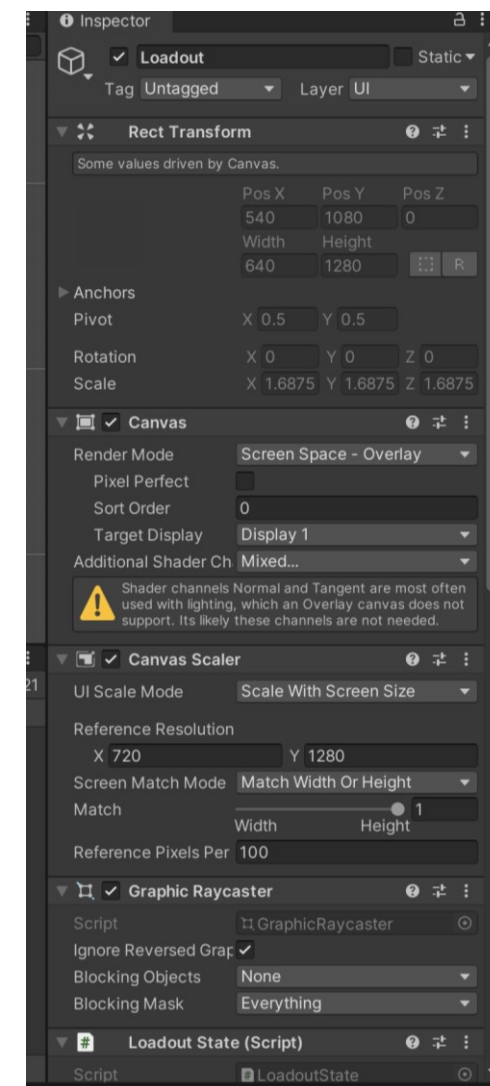
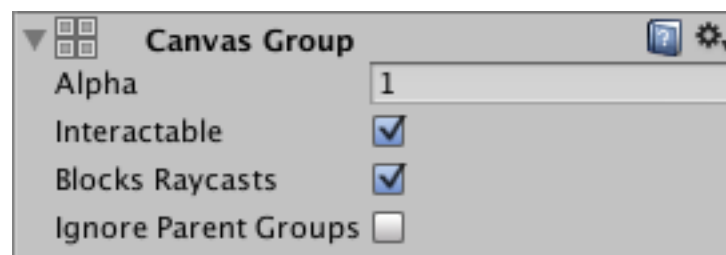
UI : componenti principali (2)

• Canvas

- La «tela» su cui potete «dipingere» la vostra GUI
- Può essere risoluzione-indipendente tramite Canvas Scaler e ancoraggi
- Parametri principali:
 - Render Mode (Overlay, World Space, Camera)
 - Sorting Layer
 - Order in layer

• Canvas Group

- Modifica la proprietà di tutti i figli
- Parametri principali:
 - Alpha
 - Interactable
 - Blocks Raycasts
 - Blocks raycast (?cos'è il raycast?)



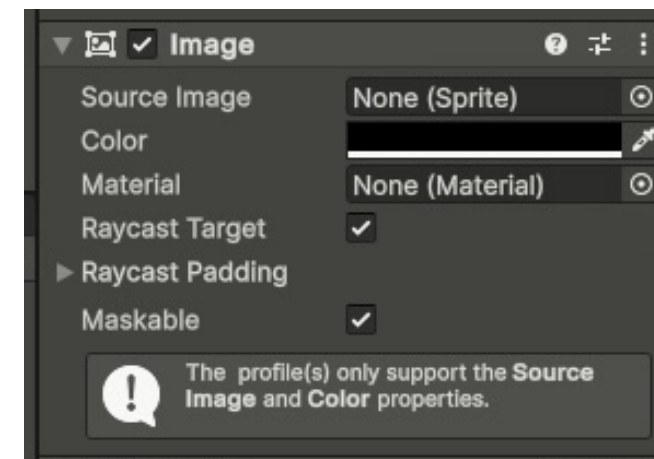
UI : componenti principali (3)

- Panel

- Raggruppa più elementi della GUI
- Tecnicamente è un semplice componente Image e il raggruppamento è dato dalla gerarchia
- Utile per fornire uno sfondo a un menù

- Image

- Una volta importata un'immagine questa può essere impostata come Sprite UI, poi assegnata al parametro Source Image
- Usata anche da altri componenti (es. Button)
- Il colore è applicato in maniera moltiplicativa (le parti nere RGB[0,0,0] non cambiano)



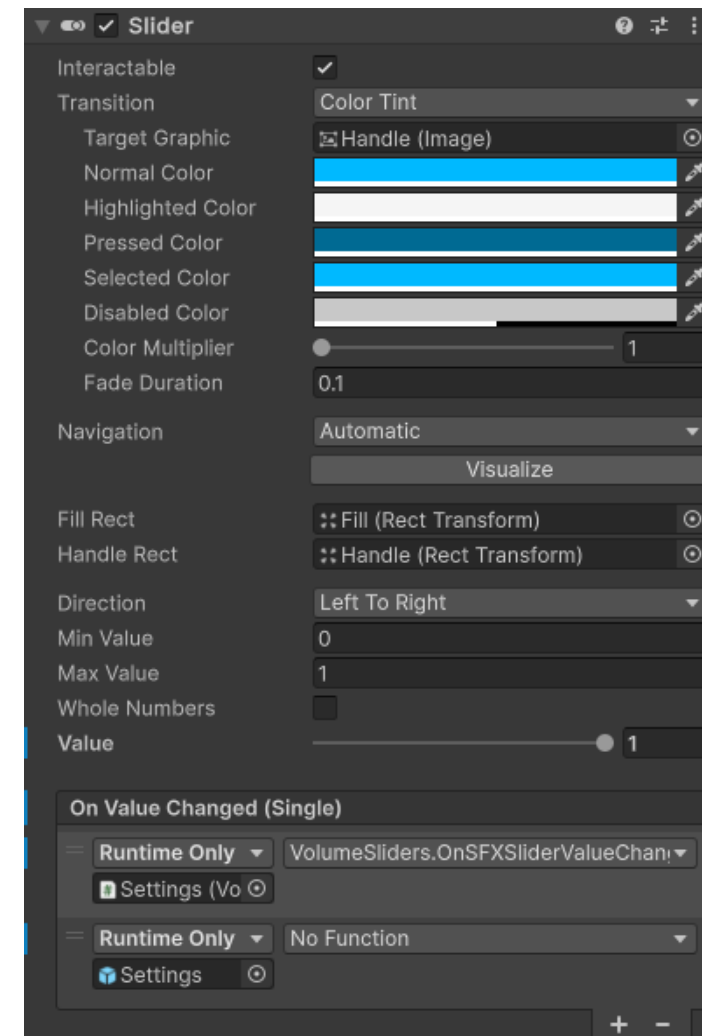
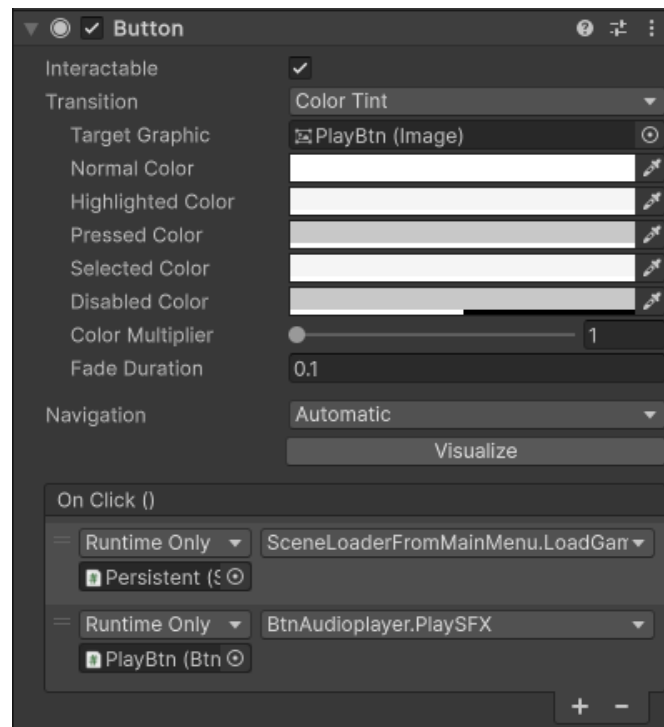
UI : componenti principali (4)

- Button

- Elemento cliccabile che lancia un evento che possiamo ascoltare via script

- Slider

- Utile per i settings e non solo (barre di caricamento, vita, ...)

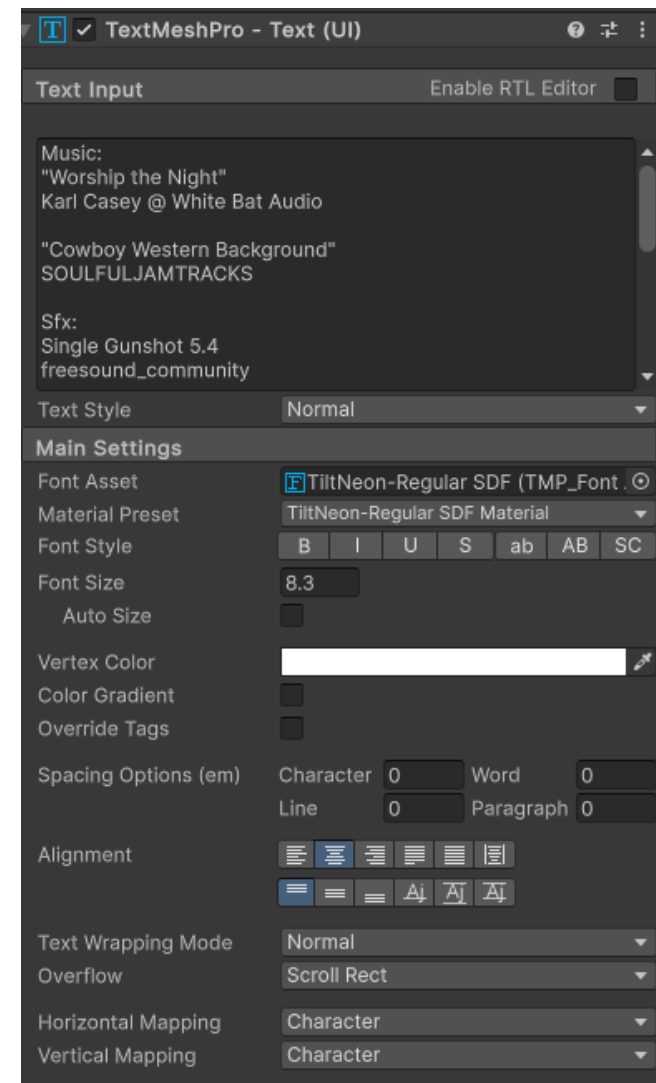
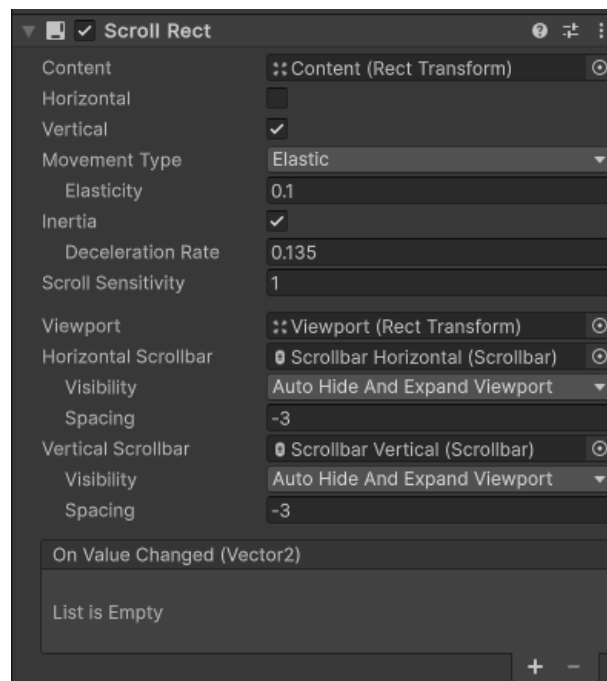


UI : componenti principali (5)

- Altri componenti di testo importanti:

- Text – TextMeshPro
- Scroll view
- Input field – TextMeshPro
-

Se avete la possibilità di scegliere fra due componenti simili, specialmente per il testo, dove c'è la dicitura «Text Mesh Pro» è sempre la scelta giusta!



Esercizio 1

Obiettivo: Produrre un menù principale (scena A) con uno sfondo, un titolo, un bottone (K) e una schermata in sovrapposizione attivabile e disattivabile con un secondo bottone (Q).

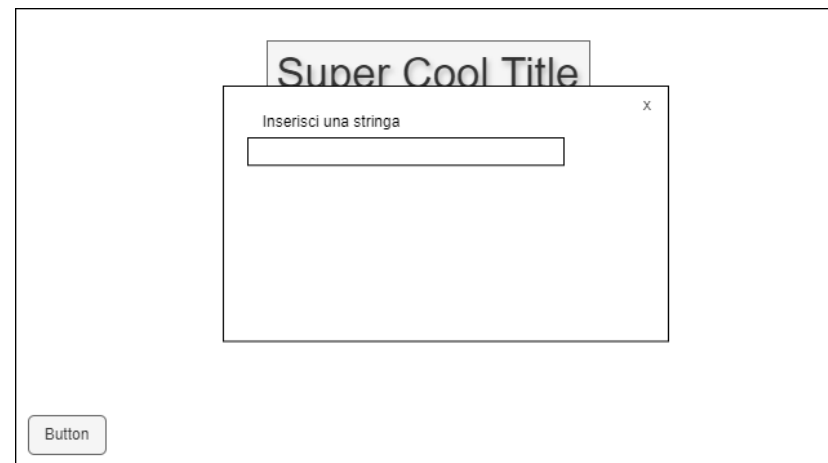
Animare il bottone K con uno script.

Premendo il bottone K, viene caricata una seconda scena (scena B).

Sulla schermata in sovrapposizione, deve esserci la possibilità di inserire una stringa di testo che deve essere passata alla scena B.

Nella scena B, impostare un campo testo con la stringa «Nel menù hai scritto: [stringa scena A]»

Suggerimento: DoNotDestroyOnLoad, SceneManager.LoadScene, InputTextField, Mathf.PingPong

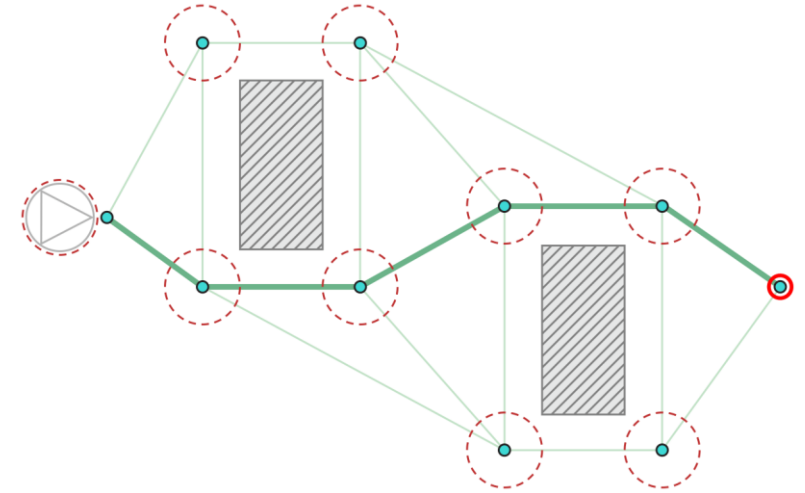




Navigazione

Navigazione: motivazioni

- Non possiamo farci carico di gestire:
 - ostacoli fissi e in movimento
 - modifiche alla topologia dell'ambiente di gioco runtime (es. mondi procedurali)
 - algoritmi di pathfinding per il calcolo di traiettorie «sensate» (tipicamente il percorso più corto per la destinazione)
 - zone calpestabili non connesse (raggiungibili con salti)
- Gli approcci visti NON scalano
 - Meno codice = meno cose che possono andare storte o rompersi = MENO BUG

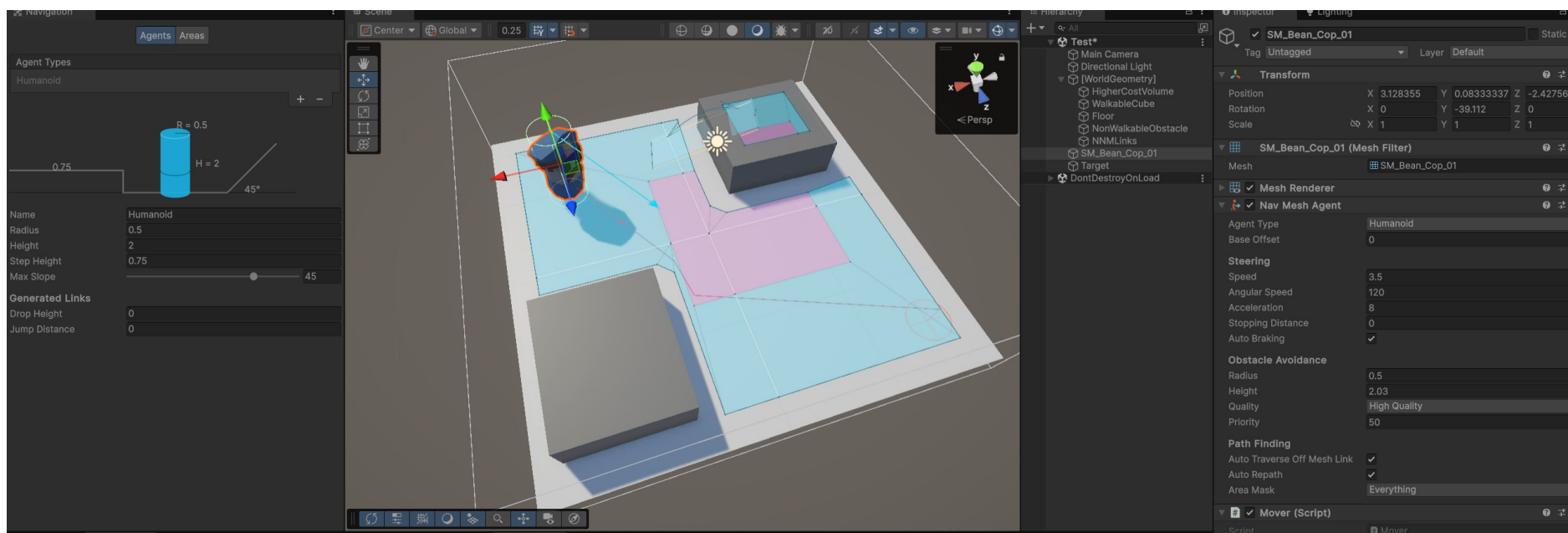


Navigazione: componenti principali (1)

- Per costruire una NM e utilizzare il sistema di navigazione nativo di Unity:

1. Individuare la geometria/fisica che influenzerà la navigazione
2. Aggiungere modificatori se necessari
3. Impostare i parametri di «baking» (Window > AI > Navigation)
4. Fare «bake» della NM

La NavMesh (NM) è la superficie attraversabile da un NMAgent



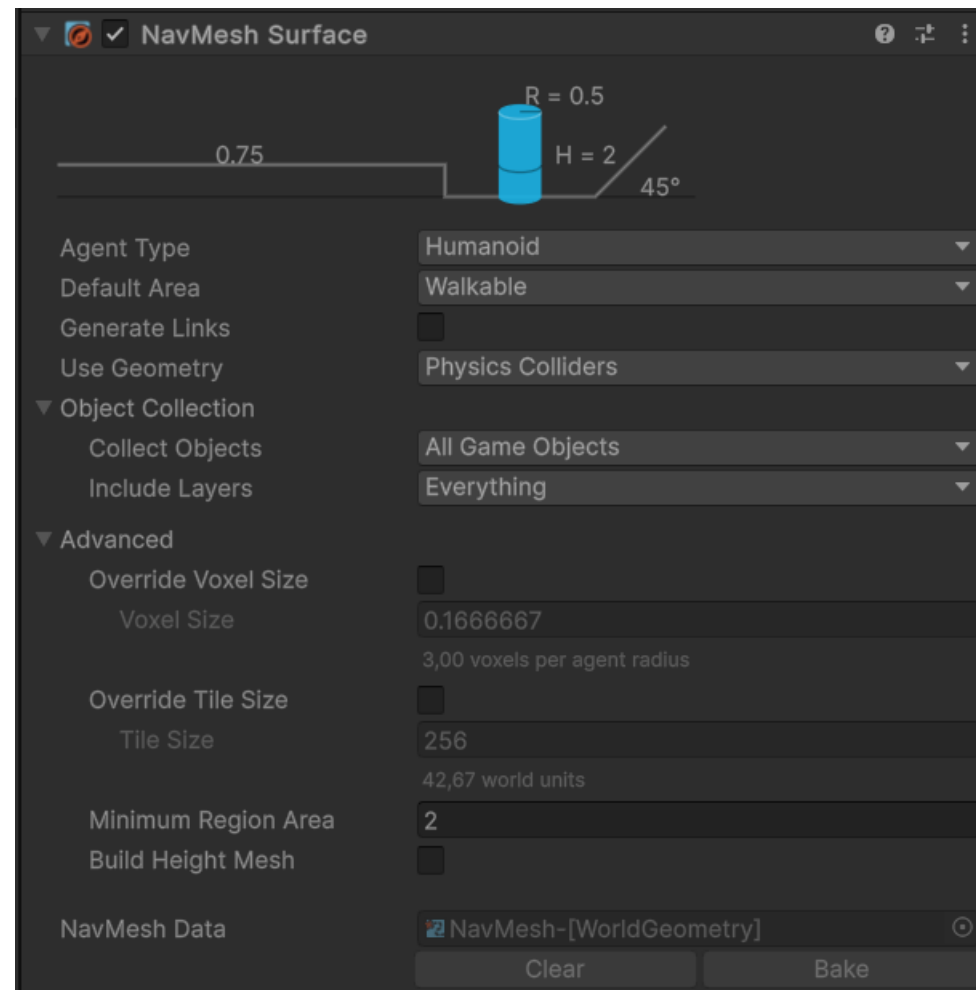
Navigazione: componenti principali (2)

- Distinguiamo i componenti per il baking e quelli (in realtà uno solo) per l'uso della NM.
- I componenti per il baking definiscono la funzione di ogni parte dell'ambiente a livello di navigazione, influenzando la costruzione (baking) della NM.

Il baking è molto simile a stendere una tovaglia su una tavola già apparecchiata e poi ritagliare buchi dove serve...

- NavMesh Surface

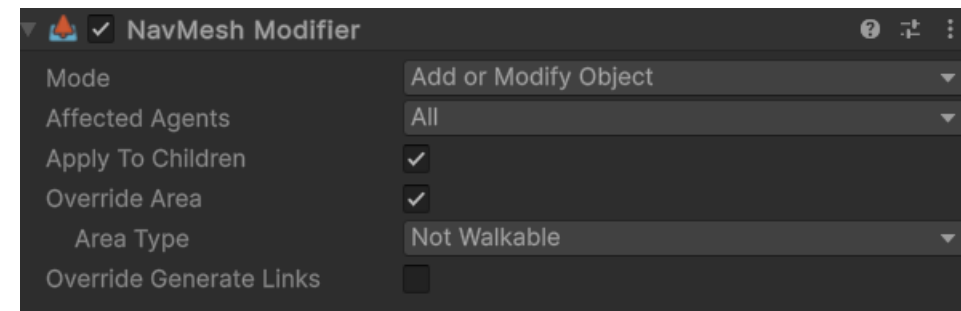
- Tutto parte da qui, definisce le proprietà e la qualità della superficie da costruire (la «tovaglia») e permette il bake
- Parametri importanti:
 - Agent Type
 - Default Area
 - Use Geometry
 - Object Collection



Navigazione: componenti principali (3)

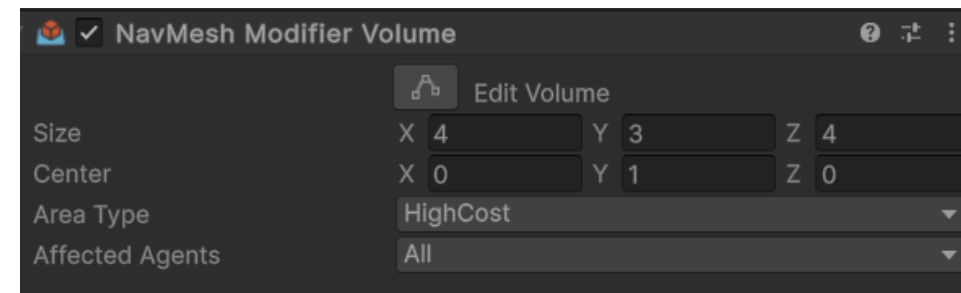
- NavMesh Modifier

- Tagga degli oggetti per modificarne il comportamento durante il bake
- Utile per marciare ostacoli che devono «bucare» la NM durante il bake
- Parametri importanti:
 - Override Area
 - Area Type



- NavMesh Modifier Volume

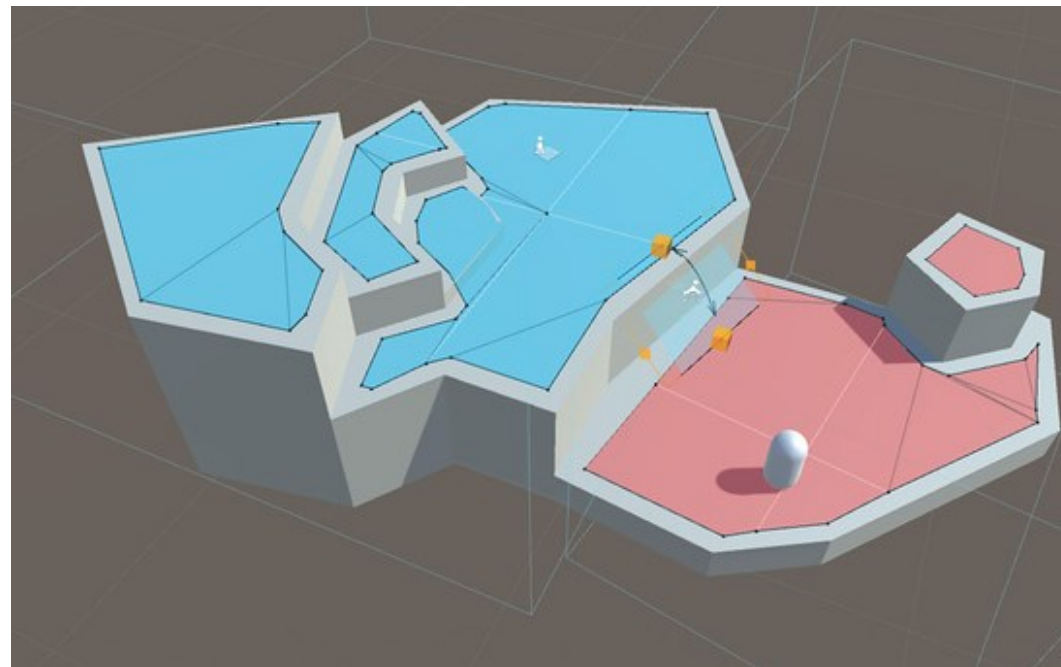
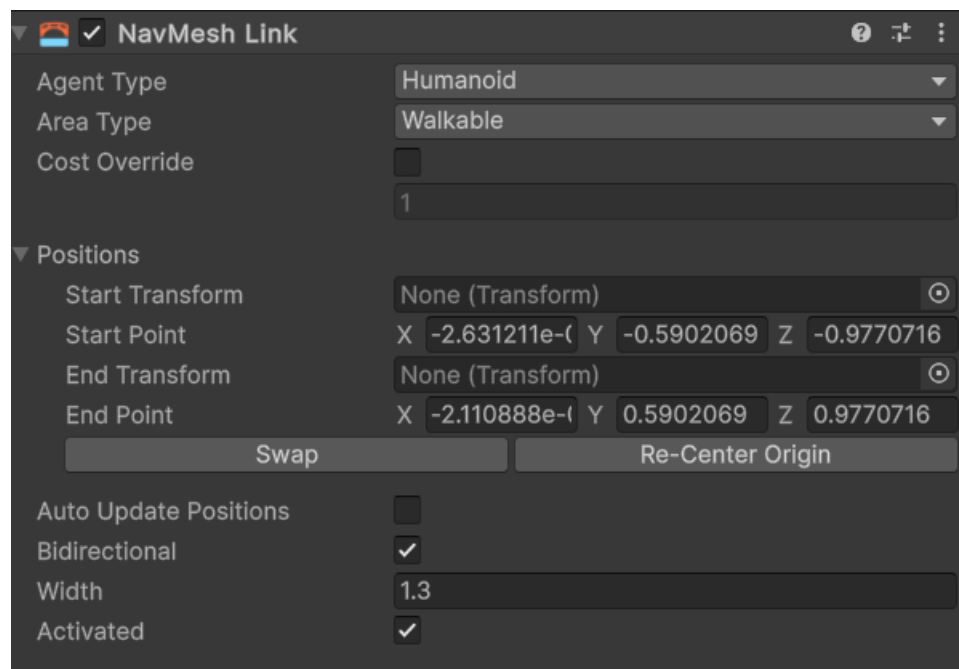
- Parente del NM Modifier
- Definisce un volume cubico in cui il baking viene influenzato, senza necessitare di geometria o collider associati
- Pro Tip: Si può usare per tagliare via parti su cui non volete la NM (es. aree non raggiungibili)



Navigazione: componenti principali (4)

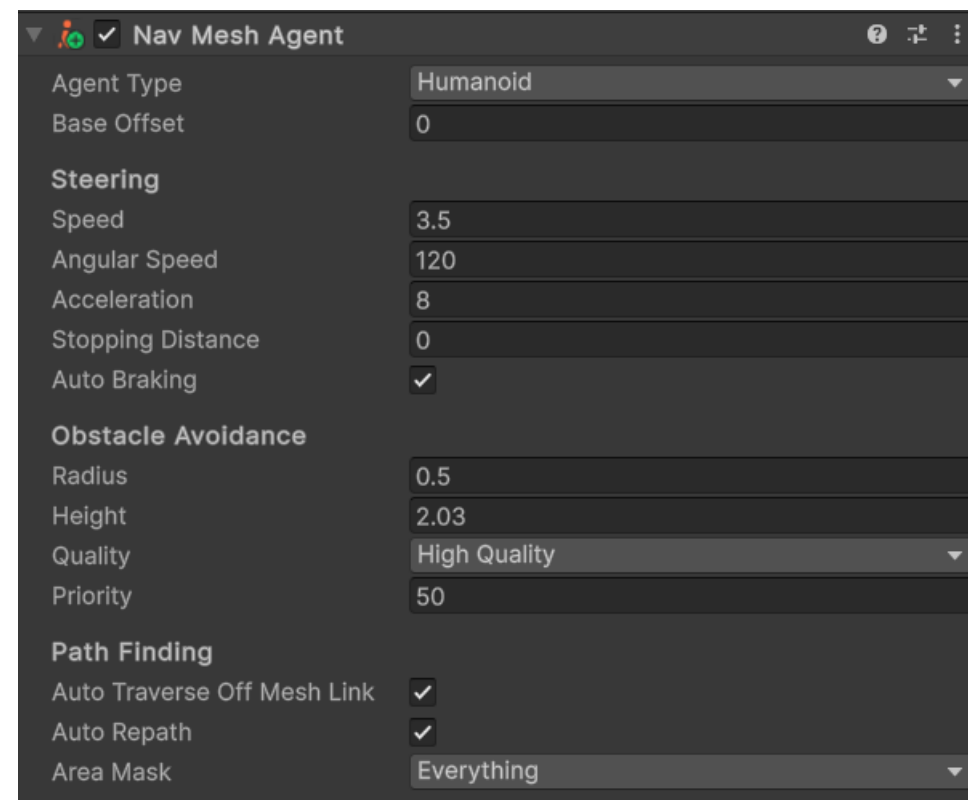
- NavMesh Link

- Crea collegamenti fra NM diverse o aree separate della stessa
- Tipicamente usati per rendere possibile arrampicarsi o salti
- Possono essere punto-punto o area-area



Navigazione: componenti principali (5)

- Per utilizzare in concreto una NM di cui avete fatto il bake si utilizza
 - NavMesh Agent
 - Definisce la rappresentazione logica di qualcosa che si muove sulla NM.
 - Parametri importanti:
 - Speed
 - Angular Speed
 - Radius
 - Priority
 - Il NM Agent ha una corposa API che vedremo rapidamente dall'editor



La NM non è utilizzabile solo lato NPC, ma anche lato player per limitare le aree accessibili

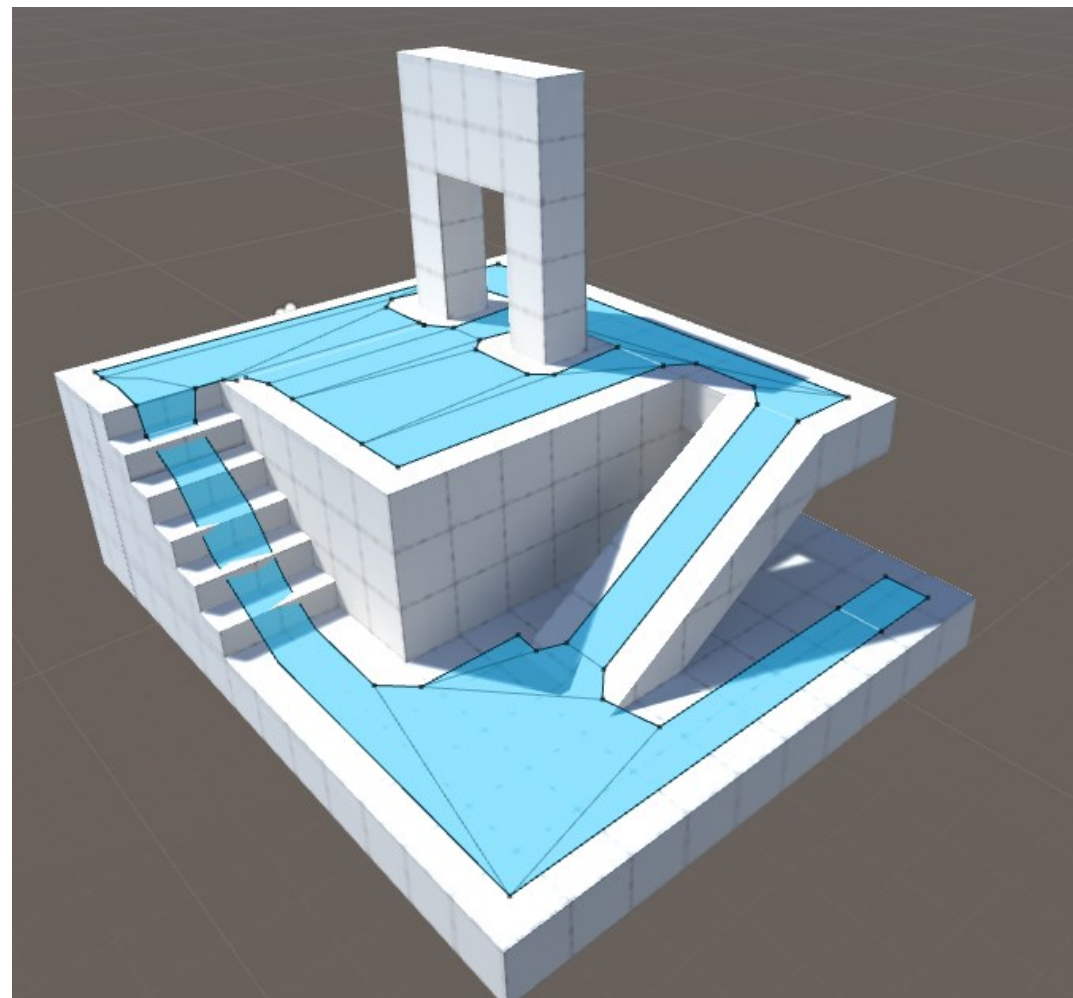
ATTENZIONE: Il NM Agent è limitato dalla NM. In particolare, non può staccarsi mai da essa (a meno dei NM Link)

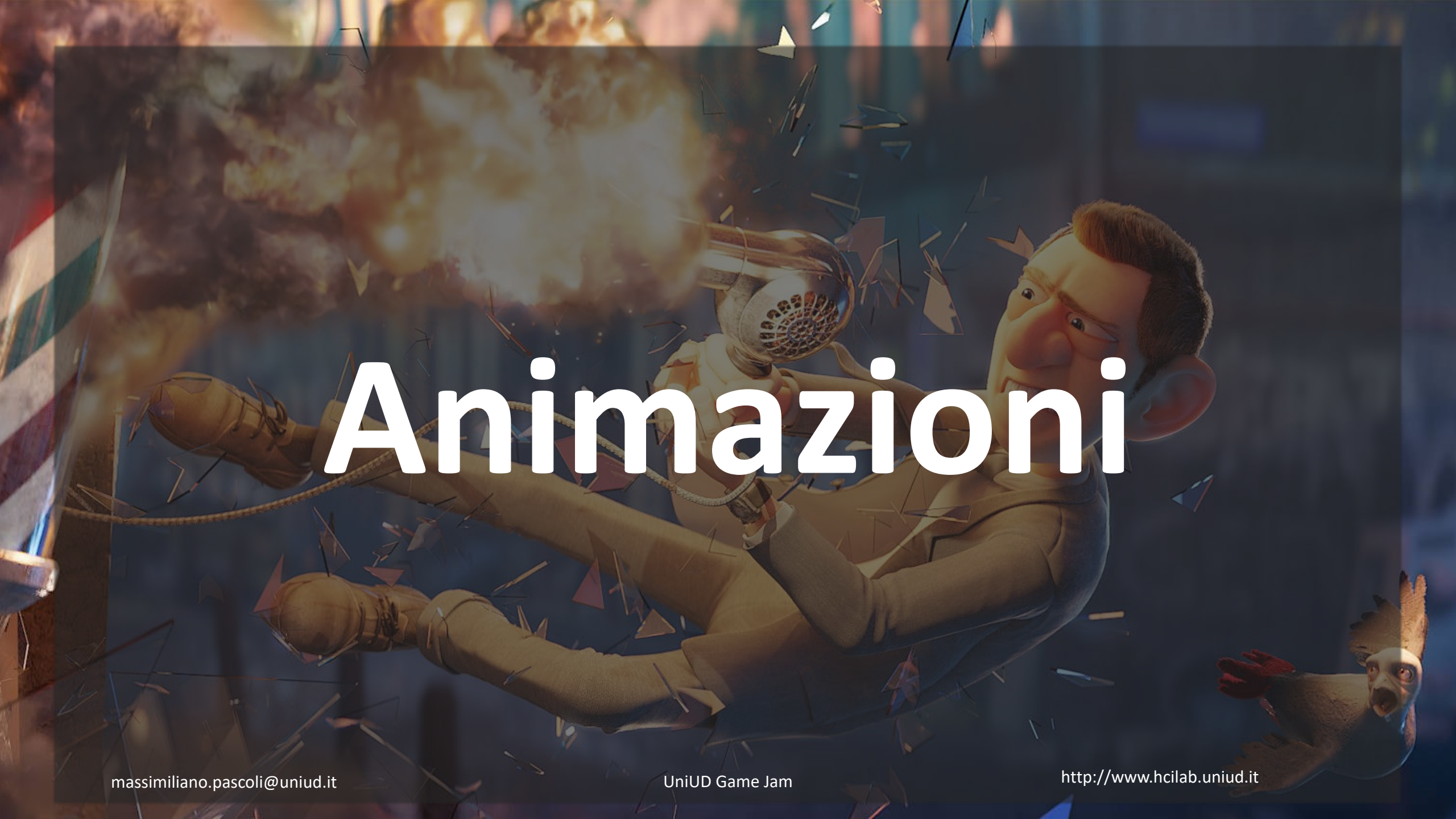
Esercizio 2

Obiettivo: Inserire un oggetto di tipo capsula che deve traslare autonomamente tra punti casuali della NM.

Suggerimento: NavMesh.SamplePosition(),
NavMeshAgent.SetDestination()

Sub-Obiettivo (lo vediamo insieme): Dotare l'AI del personaggio di una solida struttura logica, gestendo i possibili stati di un agente che utilizza le NM con una macchina a stati finiti.

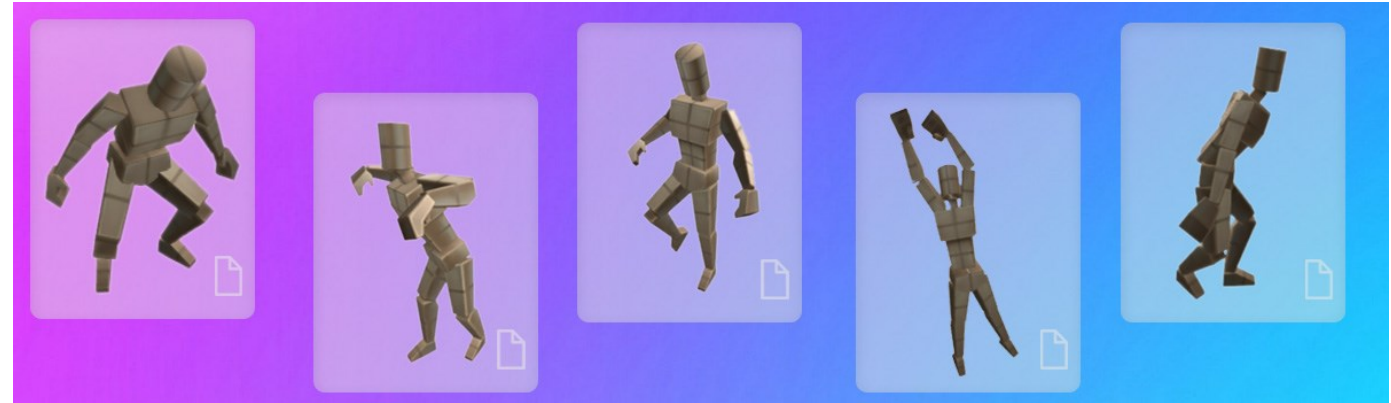




Animazioni

Animazioni: motivazioni

- Sono uno dei pilastri fondamentali di un videogioco
- Stiamo per entrare in un mondo estremamente complesso, vasto, con tematiche ancora inesplorate
- Le animazioni sono «difficili»
 - Modellazione 3D
 - Logica
 - Visualizzazione
 - Sincronizzazione
- Mostrerò solamente i concetti base



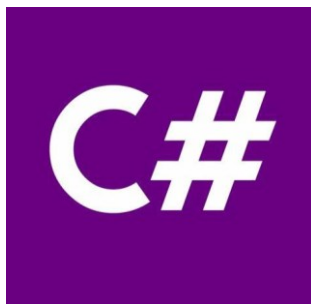
Qualsiasi cosa abbia una transform (o equivalente) o parametri esposti è animabile!

Animazioni : componenti principali (1)

- 2 modi (principali) per fare animazione:

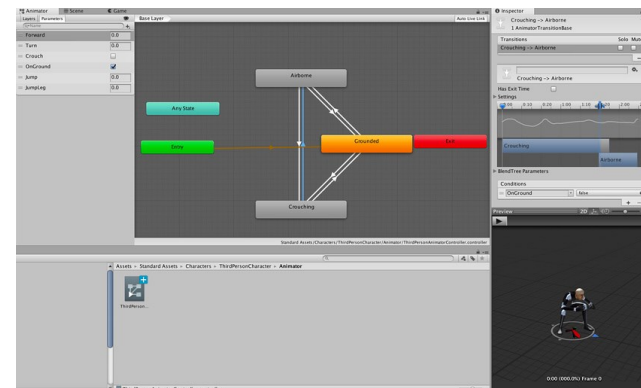
Recentemente hanno ricevuto supporto massiccio altri sistemi per gestire animazioni (e non solo): Timeline, Playables e Animation Rigging Package, ma le loro funzionalità sono vastissime e le tralasciamo.

Basata su script a basso livello



Tipicamente quando abbiamo pochi parametri da gestire (pochi = 1 transform max)

Basata su Animator



Quando abbiamo transizioni fra animazioni, sincronia fra logica e animazioni, tanti parametri

Stiamo muovendo transform o impostando parametri in maniera controllata nel tempo

Animazioni : componenti principali (2)

Script a basso livello

- I «componenti principali» dipendono da cosa volete animare

```
void PerformOpening()
{
    isInProgress = true;
    StartCoroutine(DoPerformMovement(toReach));
    Vector2 toReach = new Vector2(0f, rectTransform.anchoredPosition.y);
}

private IEnumerator DoPerformMovement(Vector2 toReach)
{
    while (Vector2.Distance(rectTransform.anchoredPosition, toReach) > 0.01f)
    {
        rectTransform.anchoredPosition = Vector2.MoveTowards(
            rectTransform.anchoredPosition, toReach, animSpeed * Time.deltaTime);
        yield return new WaitForEndOfFrame();
    }

    rectTransform.anchoredPosition = toReach;
    isInProgress = false;
    isOpen = !isOpen;
}
```

Funzioni utili:

Vector3:

- Lerp
- MoveTowards
- SmoothDamp
- RotateTowards
- Slerp

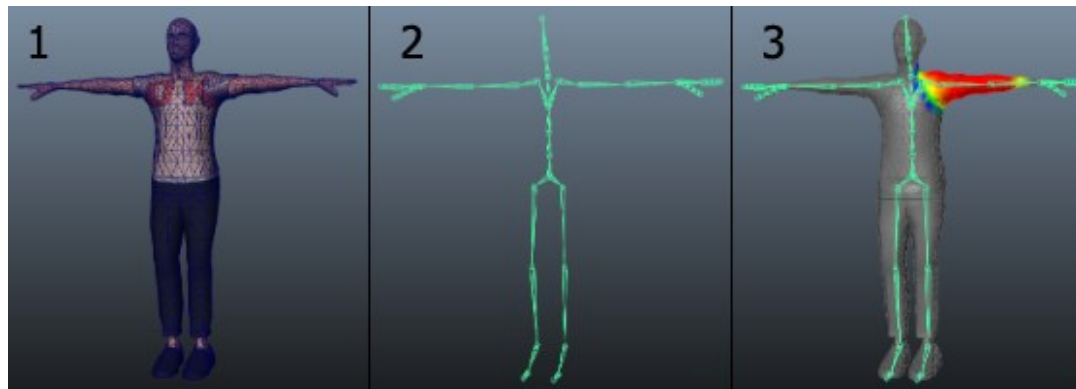
Quaternion

- Lerp
- Slerp
- Rotate towards
- AngleAxis
- FromToRotation

Animazioni : componenti principali (3)

Animator

- Sia per animazioni con scheletro (Rig) che basate su transform / parametri
- Non utilizzeremo tutto il potenziale dell'arsenale per le animazioni in Unity (troppi componenti diversi per usi specifici)

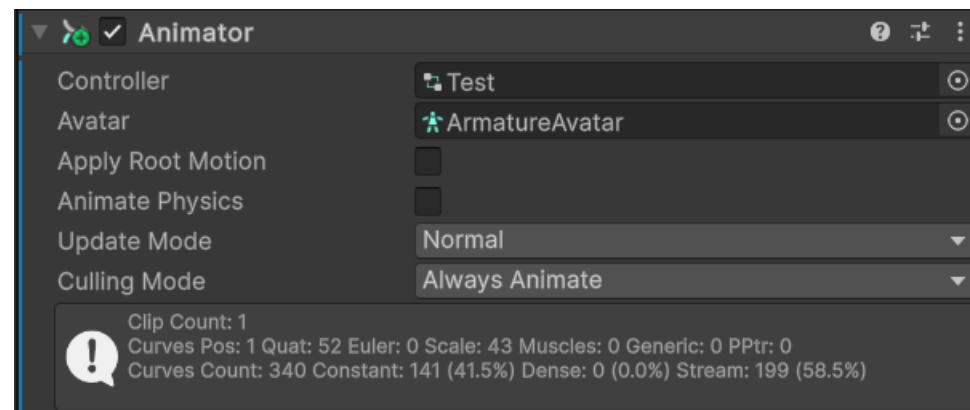


- Vedremo:
 - Animator
 - una piccola parte dell'Animator Controller
 - AnimationClips

Animazioni : componenti principali (4)

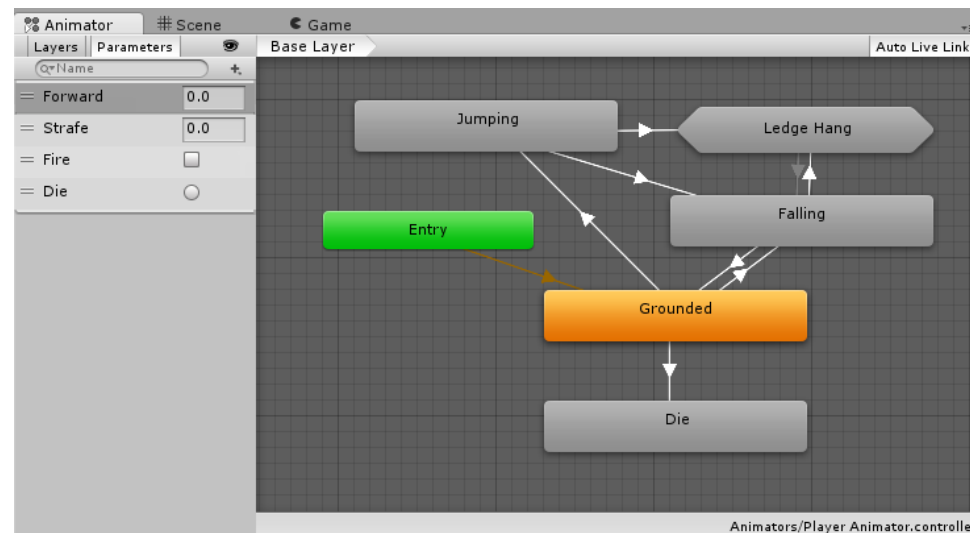
- Animator

- E' un semplice wrapper attorno ad Animation Controller
- Parametri importanti:
 - Controller
 - Avatar



- Animation Controller

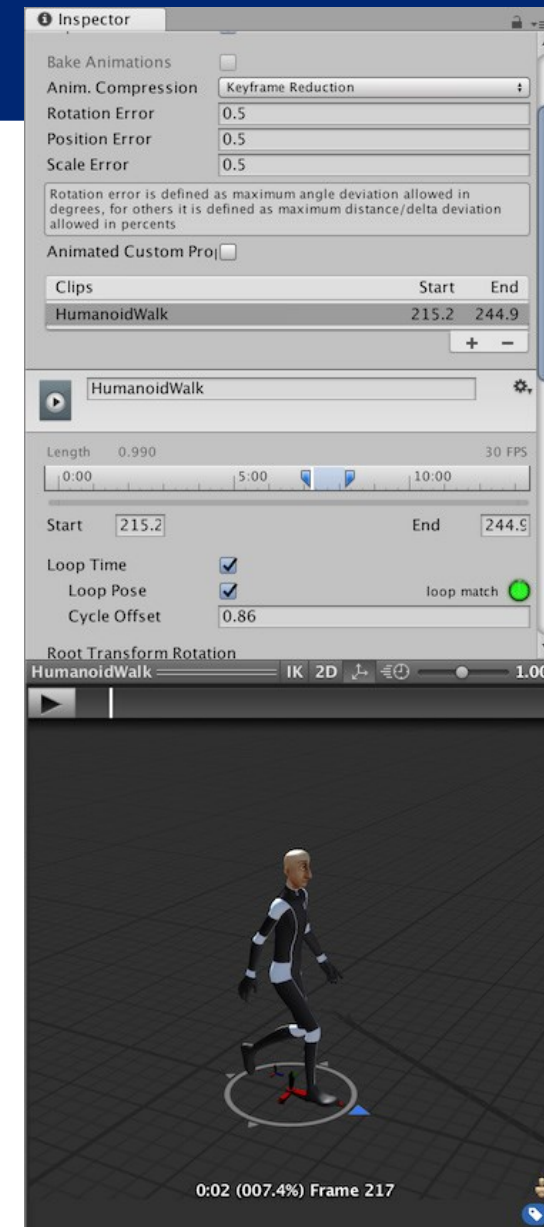
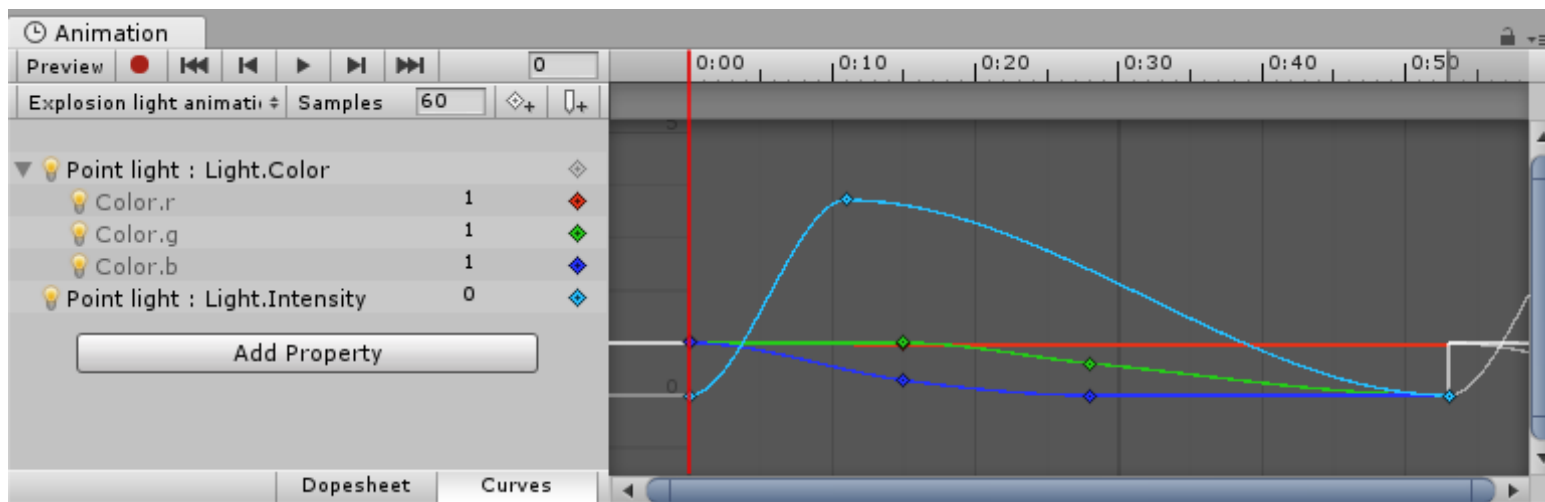
- Sostanzialmente è una macchina a stati finiti che riproduce animazioni (FK – «pre-registrate»/IK – calcolate in tempo reale) in maniera controllata e con criterio
- Gestisce transizioni fra uno stato corrente di animazione e l'altro, possibili interruzioni
- Bisogna esporre alcuni parametri per avervi accesso
- Parametri importanti:
 - Vediamo da Editor (sono parecchi)



Animazioni: componenti principali (5)

• Animation Clip

- Rappresentano come variano le posizioni delle transform o i valori dei parametri nel tempo
- Sono la registrazione di movimenti già avvenuti che possono essere riprodotti
- Parametri principali:
 - Vediamo da Editor (sono parecchi)



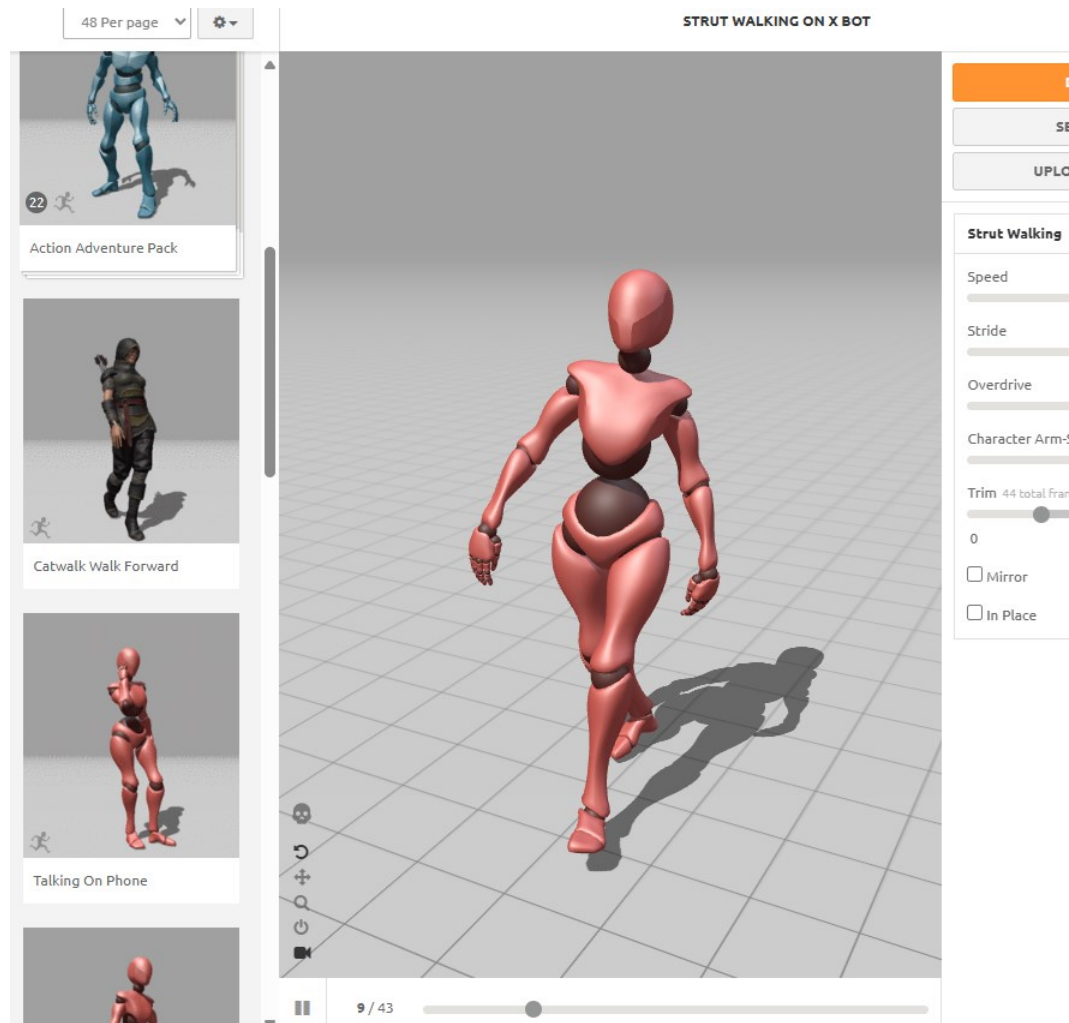
Esercizio 3

Obiettivo: Integrare un avatar base di Mixamo nella scena di gioco usando l'Animator.

Il personaggio deve camminare ed eseguire un'animazione specifica se viene premuto il tasto «E» della tastiera (scegliete voi quale).

Mentre esegue l'animazione specifica, il personaggio deve restare fermo e poi riprendere la camminata.

Suggerimento: Animator.SetTrigger(),
NavMeshAgent.isStopped



Siti utili (1)

Audio:

<https://pixabay.com/it/sound-effects/>

Icone:

<https://www.flaticon.com/>

Animazioni:

<https://www.mixamo.com/#/>

Siti utili (2)

I vostri migliori amici prima, dopo e durante la Jam:

Manuale Unity:

<https://docs.unity3d.com/6000.0/Documentation/Manual/index.html>

Scripting API Unity:

<https://docs.unity3d.com/6000.0/Documentation/ScriptReference/index.html>

